

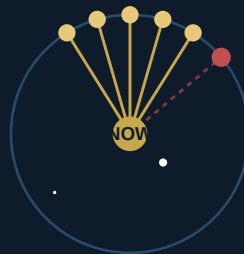
HOW NOAH KNOWS

Deciding What Is True From One Vantage Point
The Edge-Detection Epistemology Behind Every Steering Command

"We should be able to detect outliers, right?"

*"Since all edges are calculated from a single point of view —
the bottom center of the screen."*

— Scott Christopher Wilson



five witnesses agree · one does not

One Frame · Many Witnesses · Confidence, Not Certainty

SECTION 1

THE QUESTION

What Does It Mean to Know an Edge Is Real?

Noah has no ground truth. He has a depth camera, one frame at a time, and a decision to make every 250 milliseconds: where is the edge of the sidewalk, and how far should I turn to stay on it. A shadow can look like a curb. A mulch line can look like a curb. A single misread pixel, smoothed and trusted, can steer him off the real one. The question this document answers is not "how does Noah see" — that is the camera. It is **how does Noah decide what he saw was actually true**, and what he does the moment he is not sure.

"My biggest concern is false edges which are believed to be true."

— Scott Christopher Wilson

Four decisions govern this, each one a direct answer to a specific way Noah used to be fooled. None of them add a new sensor. All four come from asking sharper questions of the same single camera frame he already has.

SECTION 2

VERIFYING ACROSS MOTION

Truth Isn't in One Frame — It's in What Moves Correctly

The first version of this idea checked whether several bands of the SAME frame — near, medium, far — agreed with each other. That has a real blind spot: a shadow or a seam that happens to run roughly parallel to the curb for a few feet agrees with itself across those bands just as well as a real edge does. Same-frame agreement proves a detection looks consistent. It does not prove it is a physical, three-dimensional thing. It was also, as it turned out, wired into a detector path (`_compute_edge_guidance`) that a config flag set elsewhere made unreachable — described in an earlier draft of this document as live behavior it never actually was. Removed 2026-07-08, along with the rest of that dead branch, rather than left in place pretending to run.

The rule that replaced it

A real, physical edge is a fixed point in the world. Noah always knows how much his own heading has changed since the last tick — the compass reports it, the same way it already reports pitch and roll. So instead of asking "do nearby pixels in this one glance agree," Noah asks a stronger question: "if that edge point is real, and I know exactly how much I just turned, does it reappear exactly where physics says it must?" A shadow does not obey that rotation — its position and contrast depend on the sun angle relative to the camera, not on depth. A real curb has no choice but to match the prediction.

```
# theta = how much heading changed since the point was last accepted
pred_x = cos(theta) * prev_x - sin(theta) * prev_y
pred_y = sin(theta) * prev_x + cos(theta) * prev_y
residual = distance(this_tick_x_y, (pred_x, pred_y))
confidence *= max(0.4, 1 - residual / edge_residual_scale_m) # never raises it
```

realsense_vision.py, _predict_and_score — only evaluated when heading has changed by at least `edge_residual_min_dtheta_deg` (3°): with near-zero heading change there is no way to also separate out forward travel from a pure rotation model, so the check stays inert on ordinary straight-line driving rather than risk marking a real edge down for no reason.

This was Scott's framing, not an incremental patch on the old one: "we can yaw Noah... look for shadows changing the contrast on the sidewalk... I want truth." A deliberate verification yaw is the same test with a wider, more decisive baseline — the natural next step once this is proven on the ordinary steering corrections Noah already makes.

SECTION 3

RECOGNIZING CHANGE

A Real Jump Is Not the Same as Noise — Still Open

Noah smooths the edge position tick to tick, so that ordinary camera jitter does not make him twitch the wheel. That smoothing has a cost: a plain average cannot natively tell noise apart from a real event. When the sidewalk curves while Noah is still driving straight into it — before he has turned at all — the true edge position genuinely jumps in a single tick, and the smoothed value lags behind it for several ticks afterward. That misfire was traced exactly, tick by tick, in simulation: a commanded turn that went from thirty-four degrees to fifteen to negative fifteen to negative forty-nine across four ticks, purely because the average had not caught up to a curve that had already happened.

Honestly: not yet re-solved

A jump-reset rule for exactly this existed once (`edge_ema_reset_jump_m`: snap to a fresh reading instead of blending it, when the jump is too large to be jitter) — but it lived in `_smooth_guidance_obs`, part of the same dead branch removed 2026-07-08. It is gone now, and the live smoothing function (`_smooth_obs`) still has no jump escape. The motion-verification check in Section 2 does not cover this case either — it only activates when Noah's own heading changes, and this failure mode is specifically about a curve arriving before he has started turning at all. This page is left in the document as an open item, not deleted, so the concern it raised does not quietly disappear along with the code that once addressed it.

SECTION 4

LOOKING IN THE RIGHT PLACE

What The Live Detector Actually Scans For

An earlier draft of this document described Noah scanning for the band nearest a fixed lookahead distance, the same distance every tick, as a deliberate fix for a detector that used to grab whichever point was simply closest and available. That fix is real, but it lives in `_compute_edge_guidance` — the branch removed 2026-07-08 alongside the same-frame witness check. It was never reachable under the config this rover actually ships with.

What actually runs: `_detect_edges_hough / line_to_obs`

The live detector fits one line per side from the whole ROI, then scans that line from the bottom of the frame upward and reports the FIRST row past a minimum forward distance (`edge_lookahead_m`) with valid depth — nearest-available beyond a floor, not nearest-to-a-fixed-target. Because the line itself is fit from every valid segment in the ROI, not just the reported point, a single noisy row can't swing the whole reading the way a raw nearest-pixel pick could — the stability the old fix was reaching for comes from the whole-line fit, by a different route than originally documented here.

SECTION 5

ANCHORING TO SELF

When One Edge Is Missing, Start From Where I Am

A sidewalk's two edges run parallel, even around a curve. So when only one edge is actually visible, Noah can reasonably infer where the other one probably is, by mirroring the visible edge's angle onto the missing side. The question is what point that mirrored line should be anchored to. It used to be anchored at the corner of the camera frame — a place with no physical meaning, forcing an extreme, false slope just to reach real data a few rows away. The corrected anchor is bottom-center: Noah's own position, projected straight down. Both the real line and the inferred one now fan out from where he actually stands, not from an arbitrary corner of the picture.

SECTION 6

WHAT NOAH DOES WHEN HE IS NOT SURE

Confidence, Not Certainty

What actually shapes confidence on the live path today is smaller than a younger draft of this document claimed, and every part of it produces a number, never a flat yes or no. Distance weighting trusts a close reading more than a far one. Seeing both edges at a plausible sidewalk width boosts both sides at once — the strongest, most stable case. The missing-edge mirror (Section 5) is anchored at Noah's own position, so an inferred edge is never worse-grounded than it has to be. And now, whenever heading has genuinely changed, the motion check in Section 2 can only ever push confidence down, never up — a real edge survives it; a shadow does not.

"Since all edges are calculated from a single point of view — the bottom center of the screen — we should be able to detect outliers, right?"

— Scott Christopher Wilson

That sentence started this document, and it took a wrong turn before it reached its own answer: the first attempt looked for outliers within a single frame, which is one vantage point but one instant only. The corrected answer is one vantage point across CHANGE — Noah verifies a candidate edge against what his own known motion says must be true of it, not against its neighbors in the same glance. When nothing has moved enough to test, and nothing corroborates further than distance and both-sides-seen already account for, Noah does not pretend to know — he steers anyway, at exactly the confidence the evidence earns, and no more.

SECTION 7

ONE EDGE MISSING

Facing the Outer Edge vs. Facing the Inner Edge

When only one edge is visible, Noah does not know whether it is the inner or outer curve of a bend — he only knows *left* or *right* relative to his current heading. The rule is the same either way: aim a fixed **side_offset** in from whichever one he can see.

```
only LEFT visible: u_target = left_m - side_offset
only RIGHT visible: u_target = right_m + side_offset
x_angle_deg = atan2(-u_target, forward_m)
```

Left/right is decided fresh every tick from which side of Noah's current heading the center of curvature falls on — not carried over from the last tick. Rotate far enough and the label on a given physical edge can flip, which is exactly what a full heading sweep (below) reveals.

A full sweep, position held fixed and correct

Sweeping heading error away from the correct tangent, all the way around, on the same tight curve used throughout this investigation:

Heading error from tangent	Visible	Behavior
0° (correct)	both	x_angle = +8.9° -- the ordinary curve-anticipation nudge
0° to +25° toward outer	both	angle grows smoothly, nothing lost yet
+25° to +150° toward outer	outer only -- inner vanishes	single-edge fallback runs the whole way
+90° (pointed straight at outer edge)	outer only	x_angle = -45° -- a hard correction, sign flipped from the +60° case
-25° to -180° toward inner	both, the entire way	outer never disappears in this sweep at all

The arithmetic behind three representative headings

Heading	Visible edge, reading	u_target	x_angle_deg
Correct / tangent	both, blended	-0.047	+8.9°
+60° toward outer (moderate)	outer only, m=0.447	-0.053	+10.0°

Heading	Visible edge, reading	u_target	x_angle_deg
+90° toward outer (radial, extreme)	outer only, m=0.800	+0.300	-45.0°

Two things worth understanding, not just reading off the table

- **The asymmetry is geometric, not a bug.** The inner curve is close and small, so only a narrow cone of headings around true tangent keeps a forward band able to reach it. The outer curve is farther and bigger, so an enormous range of headings -- including pointed straight at the inner curb -- still land a forward ray on it. Drifting toward the outer edge is what costs Noah the inner edge's data; drifting toward the inner edge does not, by itself, cost him the outer edge's data at this curve tightness.

- **Watch the sign flip between +60° and +90°.** Both read "outer visible, inner missing" -- same mode, same formula -- but one produces a mild +10° correction and the other a hard -45°, the opposite direction. Nothing broke; u_target crossed zero between them because at +90° Noah is looking almost straight down the outer curb, close enough that it reads as being on the wrong side of his intended offset line, not just near it. The single-edge formula is locally sound everywhere, but its output is not monotonic in heading error once the heading points nearly straight at the curb -- a small extra rotation there can flip which way it tells him to turn.

SECTION 8

THE CARROT ANGLE

From What Noah Sees to What He Does

Every rule so far answers where the edge is. None of them, by themselves, is a steering command. The one number that actually turns Noah's wheels is **x_angle_deg** — the carrot angle — and it is built from two different kinds of knowledge at once: what the camera saw this instant, and where Noah's own body is pointed and tilted right now.

Heading is baked in before the angle exists

The camera reports a pixel. Before that pixel becomes a real-world distance, Noah rotates it by his own current pitch and roll — read from the flight controller, not assumed level — so a rover cresting a driveway lip or leaning on a cross-slope still measures the true forward and lateral distance to the edge, not a foreshortened or skewed one.

```
rolled_X = cr * cam_X - sr * cam_Y # roll-compensated lateral offset
rolled_Y = sr * cam_X + cr * cam_Y
forward_m = sp * rolled_Y + cp * depth_m # pitch-compensated forward distance
```

$$cr/sr = \cos/\sin(\text{current_roll_rad}) \cdot cp/sp = \cos/\sin(\text{current_pitch_rad})$$

Then knowledge of the sidewalk itself

Which edge to steer off is not just whichever fired this frame. The left / right / center choice passes through a hysteresis check so a borderline confidence a few percent from the switch point cannot flicker the decision every tick. And the standoff distance used when only one edge is visible is not a fixed guess — it is half of last_path_width_meters, the sidewalk width Noah last measured directly, the last time both edges were actually in view together. That is knowledge carried forward from a previous tick, not a number invented for this one.

```
target_offset = chosen_edge_m -/+ side_offset # side_offset = last_path_width_m / 2
x_angle_deg = atan2(-target_offset, forward_m)
```

From boresight-relative to a number Noah can act on

x_angle_deg says nothing about compass direction — zero means "drive straight ahead from wherever the camera is currently pointed," not north. carrot.js turns it into an actual steering command in four steps: flip it to match the chassis convention (correction_direction), zero it below edge_steer_deadband_deg so camera noise cannot twitch the wheel, rescale it from the camera's working angle range onto the servo's

real steering range, and finally ease toward that target over `steering_time_constant_s` of real wall-clock time rather than snapping to it — because the camera itself only reports 7-15 times a second depending on how busy the Pi is, so a fixed per-tick step would turn Noah at a different real-world rate depending on system load.

```
raw = (edge_guidance_valid ? x_angle_deg : map_angle_deg) * correction_direction
raw = 0 if abs(raw) < edge_steer_deadband_deg else raw
clamped = clamp(raw, -sidewalk_steer_input_max_deg, +sidewalk_steer_input_max_deg)
target = (clamped / sidewalk_steer_input_max_deg) * sidewalk_steer_max_deg

alpha = min(1, elapsed_seconds / steering_time_constant_s)
steering_angle_deg += alpha * (target - steering_angle_deg)
```

lib/yellow_brick_road/carrot.js

Where heading re-enters: the fallback memory

One more step happens back in `follow_the_yellow_brick_road.js`: the carrot angle, still boresight-relative, is meant to be added to Noah's current compass heading to produce an absolute target heading, remembered every tick as `last_known_carrot_target_hdg`. The whole point is a fallback — if the camera loses the edge completely, Noah should be able to yaw back toward roughly where the sidewalk last was using the compass instead of going in blind.

This is where the document's own rule had to apply to itself. That line used to read `white_rabbit.motor.carrot_heading_deg` — a field nothing in the codebase ever set. The carrot angle actually lives in `motor.steering_angle_deg`. `heading + undefined` was `NaN`, every tick, so `last_known_carrot_target_hdg` was not yet the memory it was written to be. Not a false edge this time — a false witness in the wiring between two files. Caught while writing this section, fixed the same day (2026-07-08): the line now reads `motor.steering_angle_deg`, the field `carrot.js` actually writes.

Everything on this page — the pitch/roll-compensated projection, the hysteresis-gated edge choice, the learned side offset, the deadband, rescale, time-constant smoothing, and now the compass-heading fallback memory — is live and correct on Noah today.

SECTION 9

REFERENCE

Status of Every Decision in This Document

Decision	Where	Status, as of 2026-07-08
Anchor to self, not the frame	<code>_detect_edges_hough</code>	LIVE. Missing-edge mirror anchors at bottom-center, not the frame corner.
Verify across motion	<code>_predict_and_score</code> (new)	LIVE. Heading-compensated reprojection residual; can only lower confidence, never raise it.
Same-frame band corroboration	<code>_corroborated_pick</code> , <code>_compute_edge_guidance</code>	REMOVED. Unreachable under any shipped config; superseded by the motion check above.
Jump-vs-noise EMA reset	<code>_smooth_guidance_obs</code>	REMOVED along with the above. STILL OPEN on the live path -- see Section 3.
"Look in the right place" (fixed lookahead band)	<code>_compute_edge_guidance</code>	REMOVED. Live detector uses a different, also-stable route -- see Section 4.
carrot_heading_deg fallback memory	<code>follow_the_yellow_brick_road.js</code>	FIXED. Read a field nothing set; now reads <code>motor.steering_angle_deg</code> . See Section 8.

Nothing in this table is aspirational. Every LIVE row is code that runs on Noah today; every REMOVED row is code that no longer exists rather than code left pretending to run; every OPEN row is a real, named gap, not a solved problem. An earlier draft of this document described three of these rows as live when they were not — caught and corrected the same day, in the same spirit the document itself argues for: a claim that cannot be checked against the actual running code is not yet knowledge.